

- Campus OS Authentication & Authorization System
 - Table of Contents
 - Overview
 - Key Features
 - Architecture
 - Service Ports
 - Authentication Flow
 - Registration Flow
 - Login Flow
 - Token Refresh Flow
 - API Request with Authentication
 - Authorization (RBAC)
 - Role Hierarchy
 - Permission Matrix
 - Permission Implementation
 - Token Management
 - Access Token Structure
 - Refresh Token Structure
 - Token Lifecycle
 - Token Revocation
 - API Reference
 - Base URL
 - Endpoints
 - POST /auth/register
 - POST /auth/login
 - POST /auth/refresh
 - POST /auth/logout
 - POST /auth/logout-all
 - GET /auth/me
 - PUT /auth/me
 - POST /auth/change-password
 - POST /auth/forgot-password
 - POST /auth/reset-password
 - GET /auth/verify
 - GET /health
 - Security Best Practices
 - Password Requirements

- [Security Headers](#)
- [Rate Limiting \(Recommended\)](#)
- [Security Checklist](#)
- [Integration Guide](#)
 - [Service Authentication Middleware](#)
 - [Client-Side Token Management](#)
- [Database Schema](#)
- [Configuration](#)
 - [Environment Variables](#)
 - [Production Recommendations](#)
- [Changelog](#)
 - [v1.0.0 \(December 2024\)](#)

Campus OS Authentication & Authorization System

Version: 1.0.0

Last Updated: December 2024

Status: Production-Ready Design

Table of Contents

1. [Overview](#)
 2. [Architecture](#)
 3. [Authentication Flow](#)
 4. [Authorization \(RBAC\)](#)
 5. [Token Management](#)
 6. [API Reference](#)
 7. [Security Best Practices](#)
 8. [Integration Guide](#)
 9. [Database Schema](#)
 10. [Configuration](#)
-

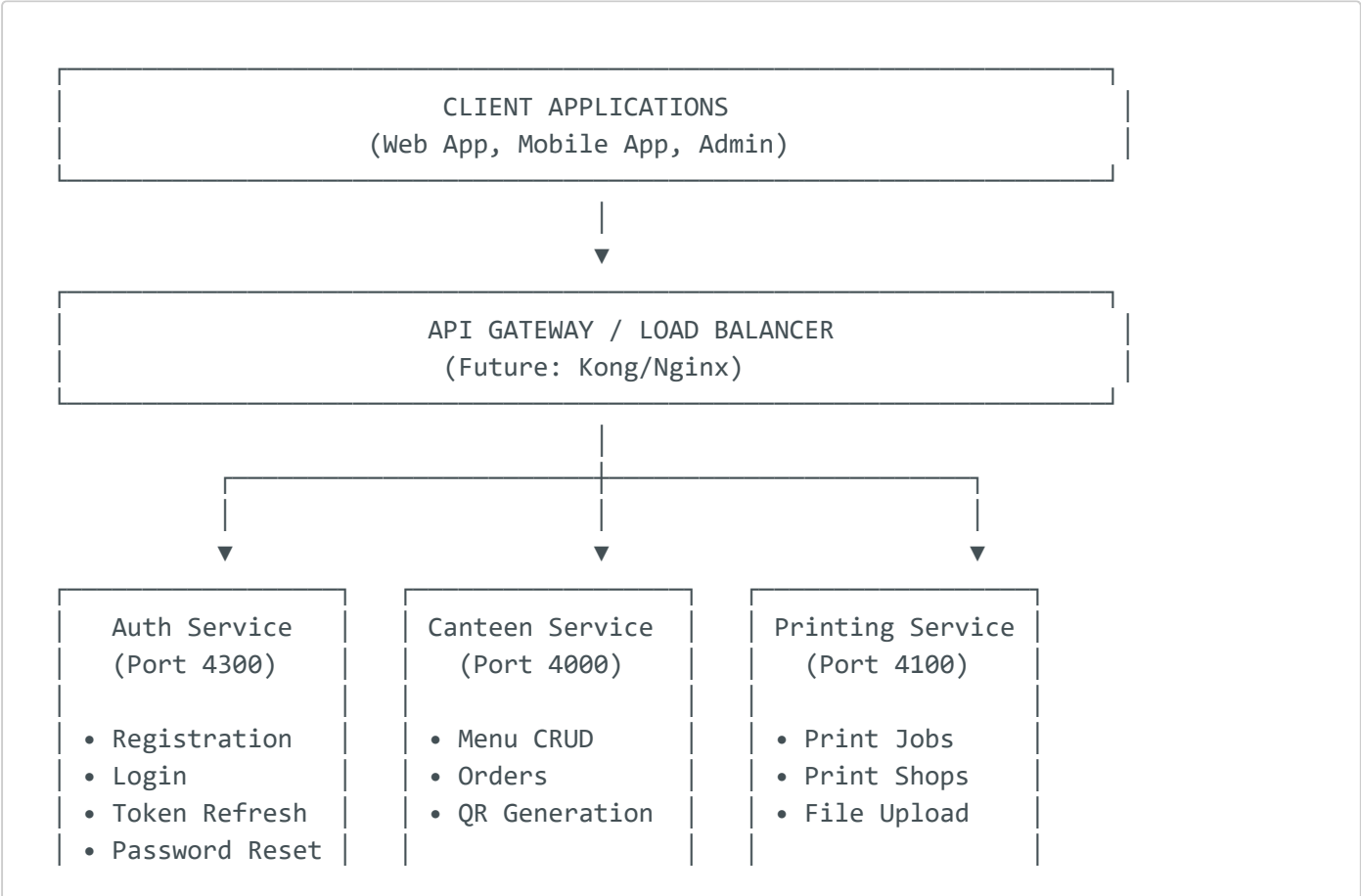
Overview

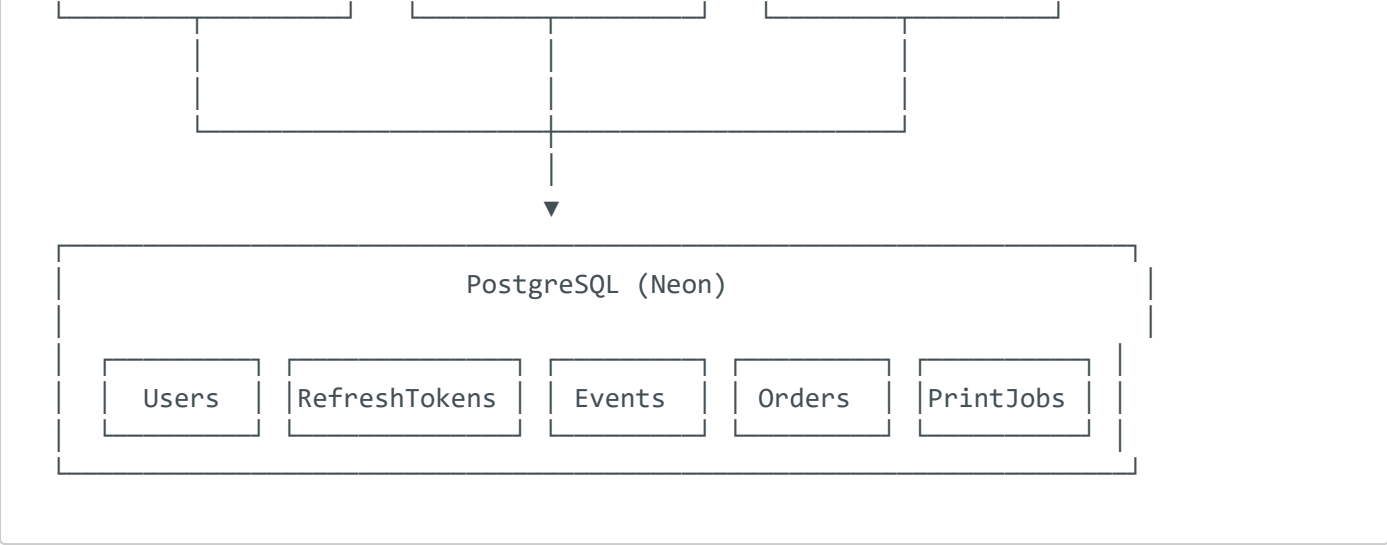
The Campus OS authentication system provides a centralized, secure authentication and authorization solution for all microservices. It uses **JWT (JSON Web Tokens)** with a dual-token strategy (access + refresh tokens) and **Role-Based Access Control (RBAC)**.

Key Features

-  **Secure Password Hashing** - bcrypt with 12 salt rounds
-  **JWT Authentication** - Short-lived access tokens (15 min)
-  **Token Refresh** - Long-lived refresh tokens (7 days) with rotation
-  **RBAC** - Four-tier role system with granular permissions
-  **Audit Logging** - All authentication events are logged
-  **Cross-Service Auth** - Single auth service for all microservices
-  **Token Revocation** - Immediate logout capability

Architecture



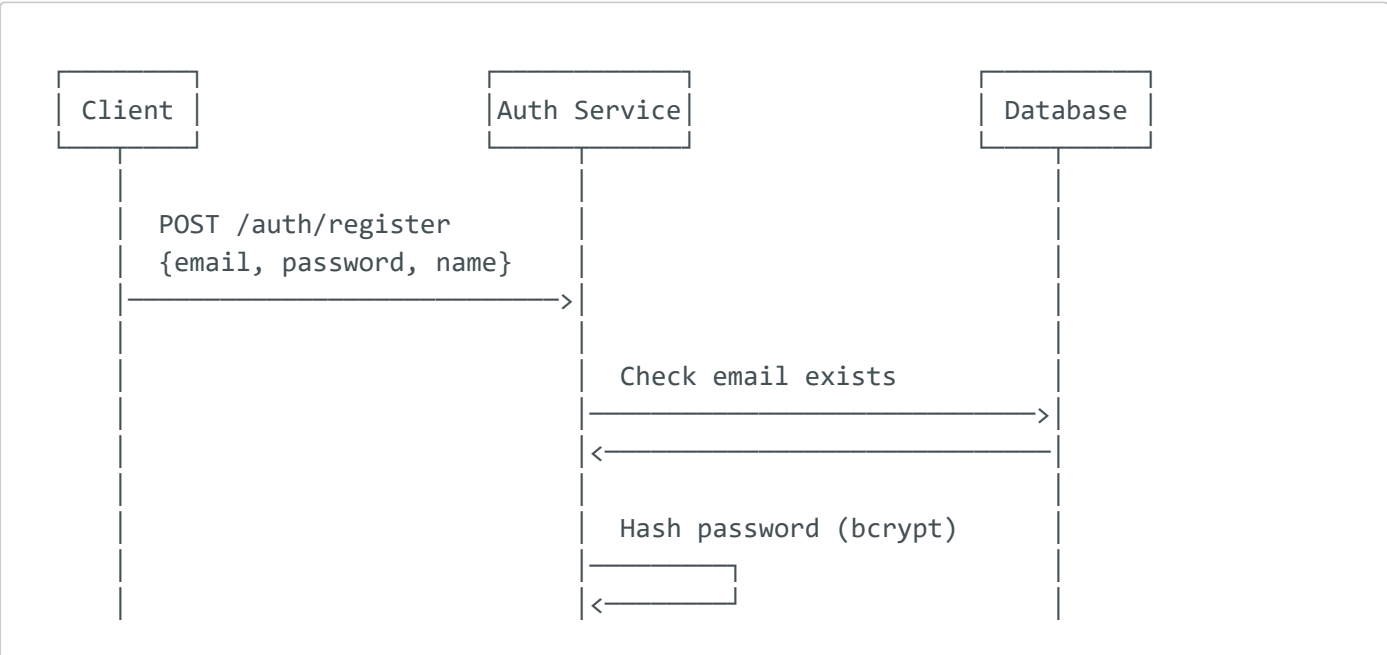


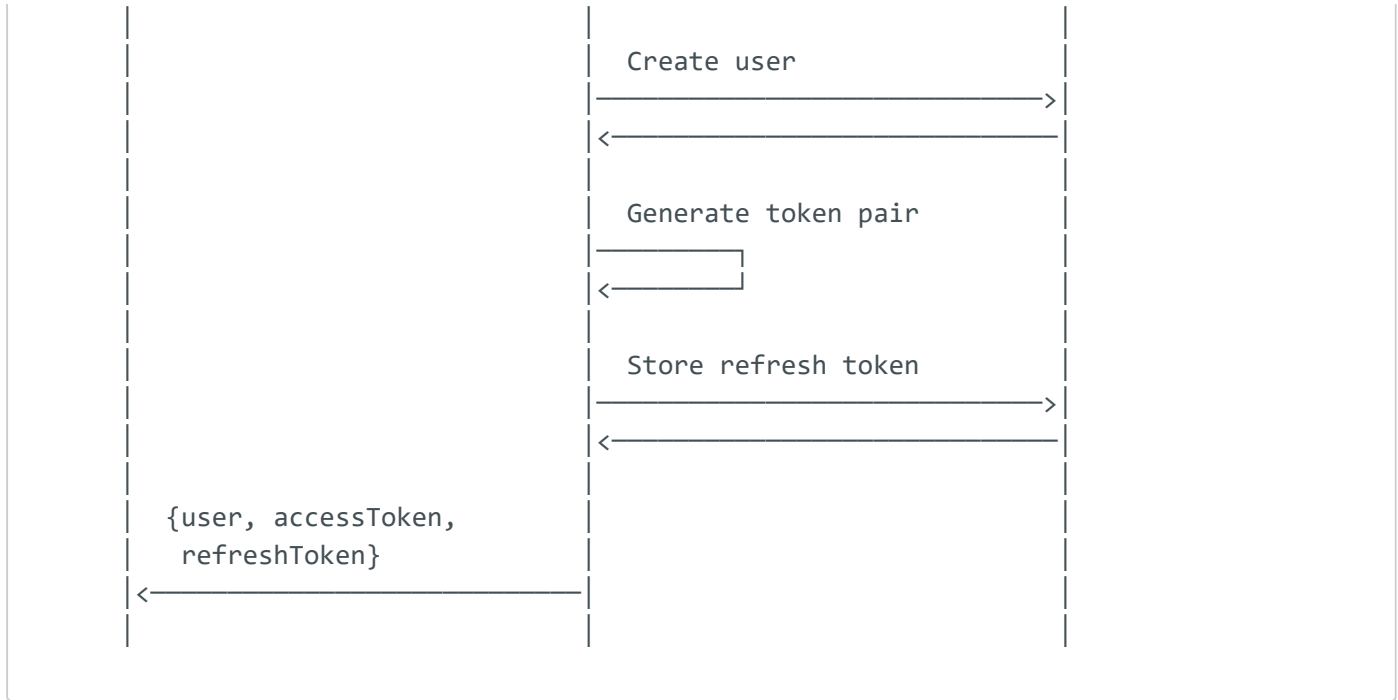
Service Ports

Service	Port	Description
Auth	4300	Authentication & Authorization
Canteen	4000	Food ordering system
Printing	4100	Document printing service
Events	4200	Campus events management

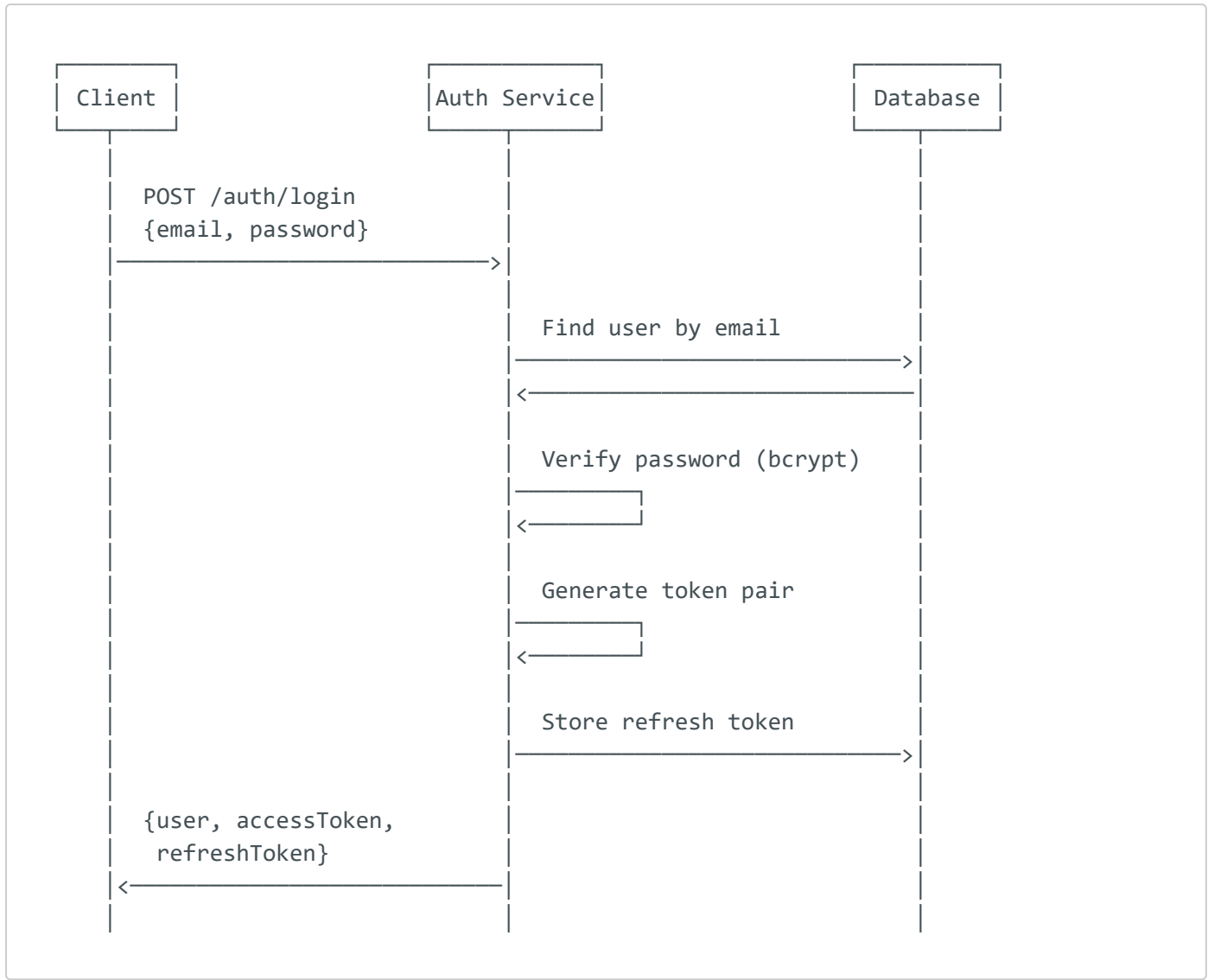
Authentication Flow

Registration Flow

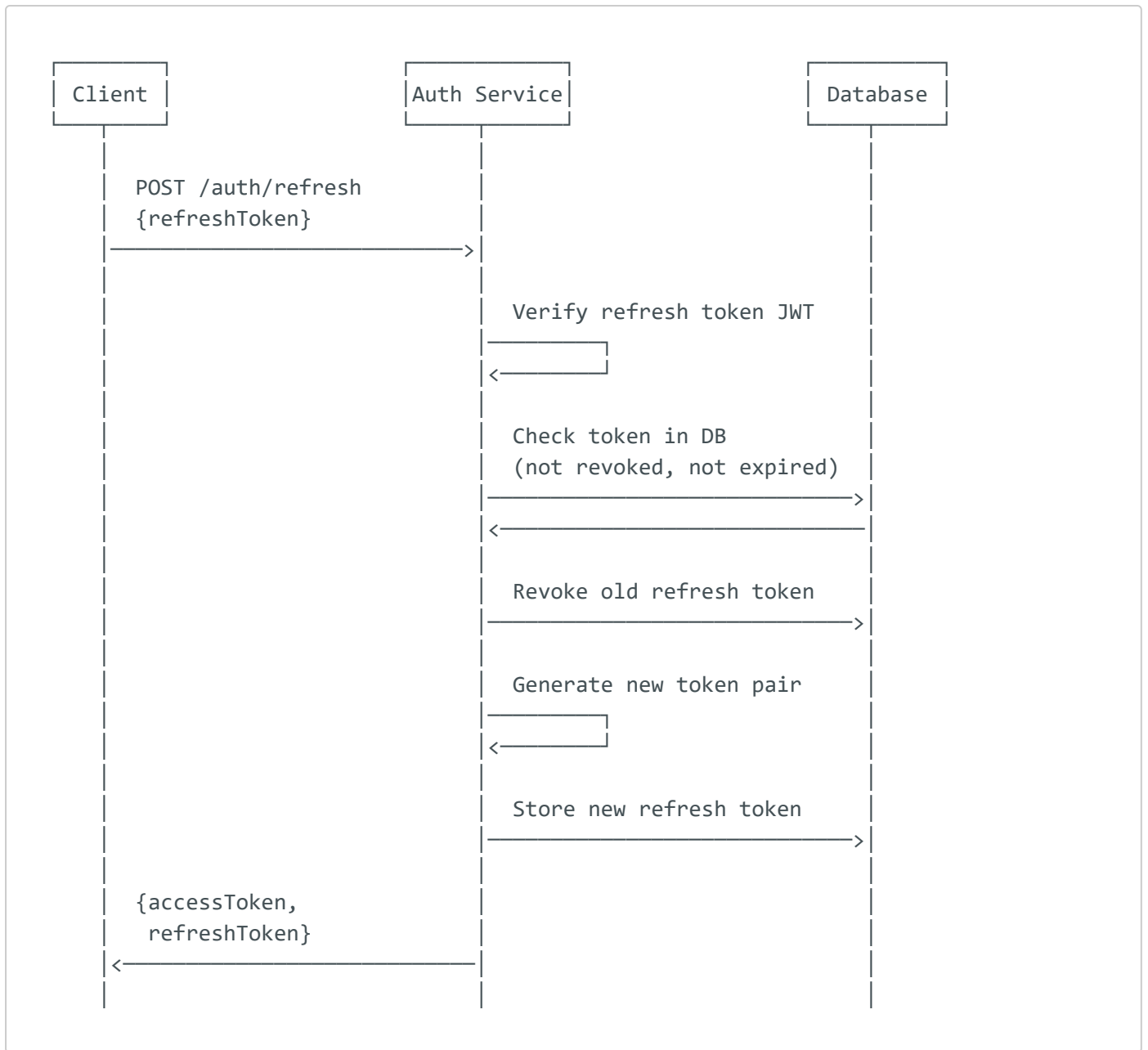




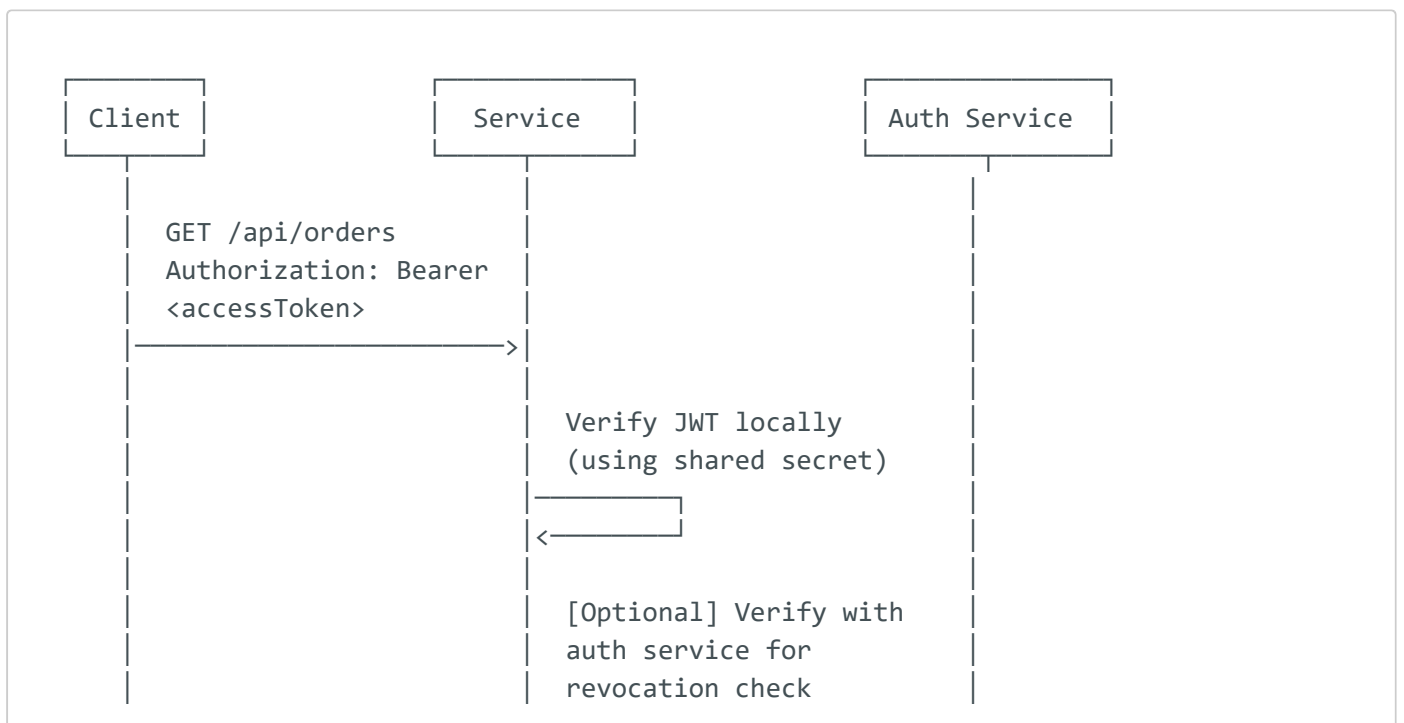
Login Flow

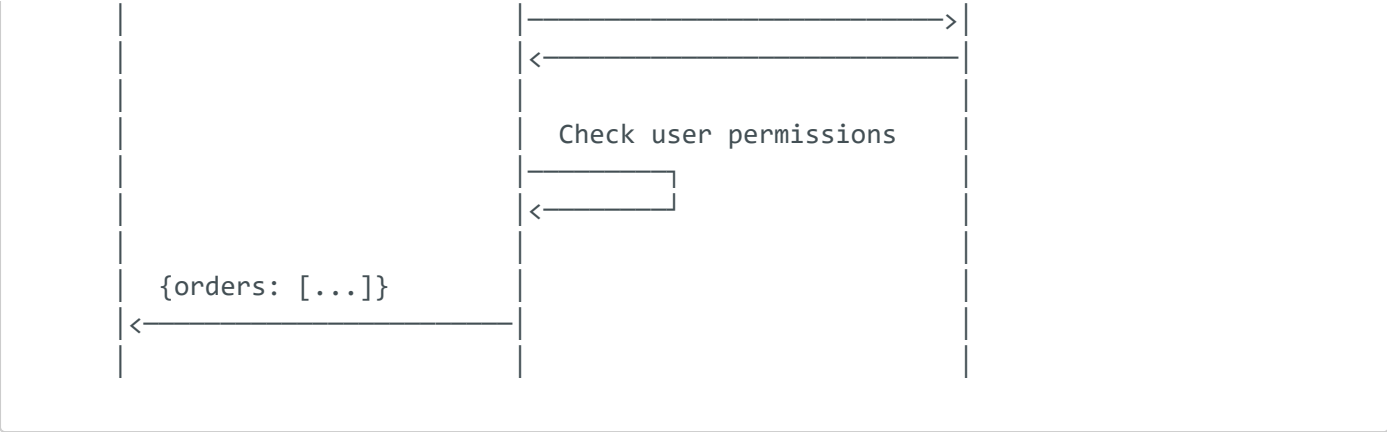


Token Refresh Flow



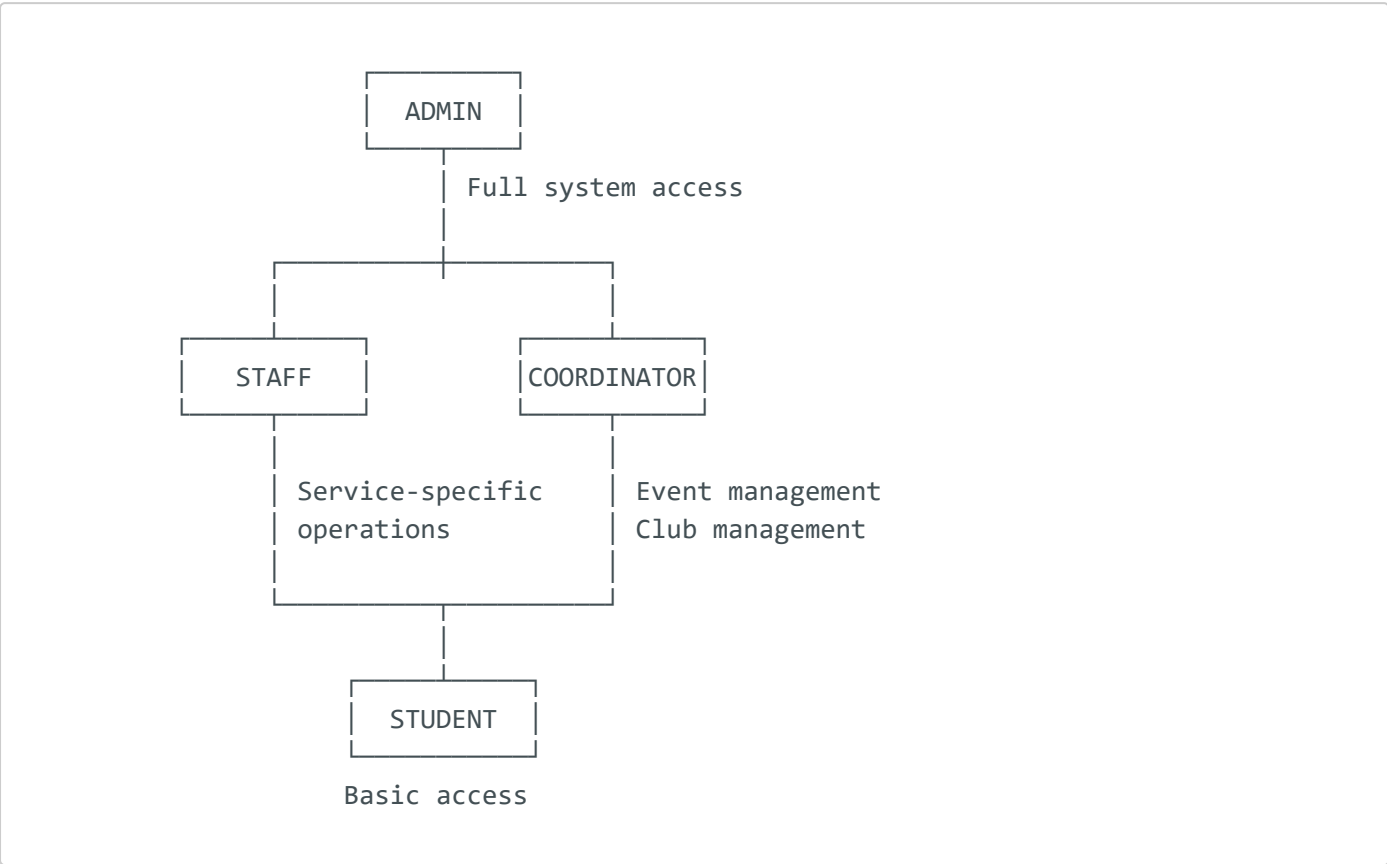
API Request with Authentication





Authorization (RBAC)

Role Hierarchy



Permission Matrix

Permission	STUDENT	STAFF	COORDINATOR	ADMIN
Canteen				
View Menu	✓	✓	✓	✓

Permission	STUDENT	STAFF	COORDINATOR	ADMIN
Place Order	✓	✓	✓	✓
View Own Orders	✓	✓	✓	✓
View All Orders	✗	✓	✗	✓
Manage Menu	✗	✓	✗	✓
Update Order Status	✗	✓	✗	✓
Printing				
Submit Print Job	✓	✓	✓	✓
View Own Print Jobs	✓	✓	✓	✓
View All Print Jobs	✗	✓	✗	✓
Manage Print Shops	✗	✓	✗	✓
Events				
View Events	✓	✓	✓	✓
Register for Events	✓	✓	✓	✓
Create Events	✗	✗	✓	✓
Manage Events	✗	✗	✓	✓
Manage Clubs	✗	✗	✓	✓
Check-in Attendees	✗	✗	✓	✓
Issue Certificates	✗	✗	✓	✓
Administration				
View Users	✗	✗	✗	✓
Manage Users	✗	✗	✗	✓
View Audit Logs	✗	✗	✗	✓
System Configuration	✗	✗	✗	✓

Permission Implementation


```
// packages/types/src/auth.ts
export type Role = 'STUDENT' | 'STAFF' | 'COORDINATOR' | 'ADMIN';

export type ModulePermission =
  // Canteen
  | 'canteen:menu:read'
  | 'canteen:menu:write'
  | 'canteen:order:create'
  | 'canteen:order:read:own'
  | 'canteen:order:read:all'
  | 'canteen:order:update'
  // Printing
  | 'printing:job:create'
  | 'printing:job:read:own'
  | 'printing:job:read:all'
  | 'printing:shop:manage'
  // Events
  | 'events:read'
  | 'events:register'
  | 'events:manage'
  | 'clubs:manage'
  | 'attendance:manage'
  | 'certificates:manage'
  // Admin
  | 'users:read'
  | 'users:manage'
  | 'audit:read';

export const ROLE_PERMISSIONS: Record<Role, ModulePermission[]> = {
  STUDENT: [
    'canteen:menu:read',
    'canteen:order:create',
    'canteen:order:read:own',
    'printing:job:create',
    'printing:job:read:own',
    'events:read',
    'events:register',
  ],
  STAFF: [
    'canteen:menu:read',
    'canteen:menu:write',
    'canteen:order:create',
    'canteen:order:read:own',
    'canteen:order:read:all',
    'canteen:order:update',
    'printing:job:create',
    'printing:job:read:own',
    'printing:job:read:all',
    'printing:shop:manage',
    'events:read',
    'events:register',
  ],
  COORDINATOR: [
    'canteen:menu:read',
    'canteen:order:create',
    'canteen:order:read:own',
  ],
}
```

```
'printing:job:create',
'printing:job:read:own',
'events:read',
'events:register',
'events:manage',
'clubs:manage',
'attendance:manage',
'certificates:manage',
],
ADMIN: [
  // All permissions
  'canteen:menu:read',
  'canteen:menu:write',
  'canteen:order:create',
  'canteen:order:read:own',
  'canteen:order:read:all',
  'canteen:order:update',
  'printing:job:create',
  'printing:job:read:own',
  'printing:job:read:all',
  'printing:shop:manage',
  'events:read',
  'events:register',
  'events:manage',
  'clubs:manage',
  'attendance:manage',
  'certificates:manage',
  'users:read',
  'users:manage',
  'audit:read',
],
};
```

Token Management

Access Token Structure

```
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },
  "payload": {
    "sub": "user-uuid",
    "email": "user@campus.edu",
    "name": "John Doe",
    "role": "STUDENT",
    "collegeId": "college-uuid",
    "type": "access",
```

```
{
  "iss": "campus-os-auth",
  "aud": "campus-os-services",
  "iat": 1703001600,
  "exp": 1703002500
}
```

Refresh Token Structure

```
{
  "header": {
    "alg": "HS256",
    "typ": "JWT"
  },
  "payload": {
    "sub": "user-uuid",
    "tokenId": "refresh-token-uuid",
    "type": "refresh",
    "iss": "campus-os-auth",
    "aud": "campus-os-services",
    "iat": 1703001600,
    "exp": 1703606400
  }
}
```

Token Lifecycle

Token Type	Validity	Storage Location	Refresh Strategy
Access Token	15 min	Memory / Secure Cookie	Auto-refresh on 401
Refresh Token	7 days	HttpOnly Cookie / DB	Rotation on each use

Token Revocation

Refresh tokens are stored in the database with a `revokedAt` field. When:

- User logs out** - Single refresh token is revoked
- User changes password** - All refresh tokens are revoked
- Admin revokes access** - All user tokens are revoked
- Token is refreshed** - Old token is revoked, new one issued

API Reference

Base URL

```
http://localhost:4300 (Development)
https://auth.campus-os.com (Production)
```

Endpoints

POST /auth/register

Create a new user account.

Request:

```
{
  "email": "student@campus.edu",
  "password": "SecureP@ss123",
  "name": "John Doe",
  "rollNumber": "2024CS001",
  "collegeId": "uuid",
  "role": "STUDENT"
}
```

Response (201):

```
{
  "user": {
    "id": "uuid",
    "email": "student@campus.edu",
    "name": "John Doe",
    "role": "STUDENT",
    "collegeId": "uuid",
    "rollNumber": "2024CS001"
  },
  "tokens": {
    "accessToken": "eyJhbG...",
    "refreshToken": "eyJhbG...",
    "accessTokenExpiresAt": "2024-01-01T00:15:00.000Z",
    "refreshTokenExpiresAt": "2024-01-08T00:00:00.000Z"
  }
}
```

```
}  
}
```

POST /auth/login

Authenticate a user.

Request:

```
{  
  "email": "student@campus.edu",  
  "password": "SecureP@ss123"  
}
```

Response (200):

```
{  
  "user": { ... },  
  "tokens": { ... }  
}
```

POST /auth/refresh

Get new access token using refresh token.

Request:

```
{  
  "refreshToken": "eyJhbG..."  
}
```

Response (200):

```
{  
  "user": { ... },  
  "tokens": { ... }  
}
```

POST /auth/logout

Logout user (revoke refresh token).

Request:

```
{
  "refreshToken": "eyJhbG..."
}
```

Response (200):

```
{
  "message": "Logged out successfully"
}
```

POST /auth/logout-all

Requires Authentication

Logout from all devices.

Response (200):

```
{
  "message": "Logged out from all devices"
}
```

GET /auth/me

Requires Authentication

Get current user profile.

Response (200):

```
{
  "id": "uuid",
  "email": "student@campus.edu",
  "name": "John Doe",
  "role": "STUDENT",
  "collegeId": "uuid",
  "rollNumber": "2024CS001",
  "college": {
```

```
    "id": "uuid",
    "name": "Engineering College",
    "code": "ENG"
  },
  "createdAt": "2024-01-01T00:00:00.000Z",
  "updatedAt": "2024-01-01T00:00:00.000Z"
}
```

PUT /auth/me

Requires Authentication

Update user profile.

Request:

```
{
  "name": "John Smith",
  "rollNumber": "2024CS002"
}
```

POST /auth/change-password

Requires Authentication

Change user password.

Request:

```
{
  "currentPassword": "OldP@ss123",
  "newPassword": "NewP@ss456"
}
```

POST /auth/forgot-password

Request password reset link.

Request:

```
{
  "email": "student@campus.edu"
}
```

```
}
```

POST /auth/reset-password

Reset password with token.

Request:

```
{
  "token": "reset-token-from-email",
  "newPassword": "NewP@ss789"
}
```

GET /auth/verify

Requires Authentication

Verify access token validity.

Response (200):

```
{
  "valid": true,
  "user": {
    "id": "uuid",
    "email": "student@campus.edu",
    "name": "John Doe",
    "role": "STUDENT",
    "collegeId": "uuid"
  }
}
```

GET /health

Health check endpoint.

Response (200):

```
{
  "status": "healthy",
  "service": "auth",
}
```



```
"timestamp": "2024-01-01T00:00:00.000Z"
}
```

Security Best Practices

Password Requirements

- Minimum 8 characters
- Maximum 128 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one number

Security Headers

All responses include:

```
Access-Control-Allow-Origin: <configured-origins>
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
```

Rate Limiting (Recommended)

Endpoint	Limit
/auth/login	5/min per IP
/auth/register	3/min per IP
/auth/forgot-password	3/min per email
/auth/refresh	10/min per user
Other endpoints	100/min per user

Security Checklist

- ☒ Passwords hashed with bcrypt (12 rounds)
 - ☒ JWT with HS256 signing
 - ☒ Short-lived access tokens
 - ☒ Refresh token rotation
 - ☒ Token revocation support
 - ☒ Input validation with Zod
 - ☒ CORS configuration
 - ☒ Structured logging
 - ☐ Rate limiting (implement with Redis)
 - ☐ HTTPS only (production)
 - ☐ Helmet.js security headers
 - ☐ SQL injection prevention (Prisma)
 - ☐ XSS prevention
 - ☐ CSRF protection
-

Integration Guide

Service Authentication Middleware

```
// services/canteen/src/infrastructure/auth/middleware.ts
import { verifyAccessToken, extractBearerToken } from '@campus-os/service-auth';
import type { AccessTokenPayload } from '@campus-os/service-auth';
import type { IncomingMessage, ServerResponse } from 'http';

export async function authenticate(
  req: IncomingMessage
): Promise<AccessTokenPayload | null> {
  const token = extractBearerToken(req.headers.authorization);
  if (!token) return null;
  return verifyAccessToken(token);
}

export function requireAuth(
  handler: (req: IncomingMessage, res: ServerResponse, user: AccessTokenPayload) =>
  Promise<void>
) {
  return async (req: IncomingMessage, res: ServerResponse) => {
    const user = await authenticate(req);
    if (!user) {
```

```

        res.writeHead(401, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ error: 'Authentication required' }));
        return;
    }
    return handler(req, res, user);
};
}

export function requireRole(roles: Role[]) {
    return (
        handler: (req: IncomingMessage, res: ServerResponse, user: AccessTokenPayload)
=> Promise<void>
    ) => {
        return requireAuth(async (req, res, user) => {
            if (!roles.includes(user.role)) {
                res.writeHead(403, { 'Content-Type': 'application/json' });
                res.end(JSON.stringify({ error: 'Insufficient permissions' }));
                return;
            }
            return handler(req, res, user);
        });
    };
}

```

Client-Side Token Management

```

// apps/web/src/lib/auth.ts
interface TokenStorage {
    accessToken: string | null;
    refreshToken: string | null;
    accessTokenExpiresAt: Date | null;
}

class AuthClient {
    private tokens: TokenStorage = {
        accessToken: null,
        refreshToken: null,
        accessTokenExpiresAt: null,
    };

    async login(email: string, password: string): Promise<User> {
        const response = await fetch('/api/auth/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ email, password }),
        });

        if (!response.ok) {
            throw new Error('Login failed');
        }

        const data = await response.json();
    }
}

```

```

    this.setTokens(data.tokens);
    return data.user;
}

async fetch(url: string, options: RequestInit = {}): Promise<Response> {
    // Check if token needs refresh
    if (this.isTokenExpiring()) {
        await this.refreshTokens();
    }

    const response = await fetch(url, {
        ...options,
        headers: {
            ...options.headers,
            Authorization: `Bearer ${this.tokens.accessToken}`,
        },
    });

    // Handle 401 by trying to refresh
    if (response.status === 401) {
        await this.refreshTokens();
        return fetch(url, {
            ...options,
            headers: {
                ...options.headers,
                Authorization: `Bearer ${this.tokens.accessToken}`,
            },
        });
    }

    return response;
}

private isTokenExpiring(): boolean {
    if (!this.tokens.accessTokenExpiresAt) return true;
    // Refresh if less than 1 minute remaining
    return new Date() >= new Date(this.tokens.accessTokenExpiresAt.getTime() -
60000);
}

private async refreshTokens(): Promise<void> {
    if (!this.tokens.refreshToken) {
        throw new Error('No refresh token');
    }

    const response = await fetch('/api/auth/refresh', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ refreshToken: this.tokens.refreshToken }),
    });

    if (!response.ok) {
        this.clearTokens();
        throw new Error('Token refresh failed');
    }

    const data = await response.json();

```

```

    this.setTokens(data.tokens);
  }

  private setTokens(tokens: any): void {
    this.tokens = {
      accessToken: tokens.accessToken,
      refreshToken: tokens.refreshToken,
      accessTokenExpiresAt: new Date(tokens.accessTokenExpiresAt),
    };
    // Optionally store refresh token in localStorage for persistence
    localStorage.setItem('refreshToken', tokens.refreshToken);
  }

  private clearTokens(): void {
    this.tokens = {
      accessToken: null,
      refreshToken: null,
      accessTokenExpiresAt: null,
    };
    localStorage.removeItem('refreshToken');
  }
}

export const auth = new AuthClient();

```

Database Schema

```

model User {
  id          String      @id @default(uuid())
  email       String      @unique
  passwordHash String
  name        String
  role        Role        @default(STUDENT)
  rollNumber  String?
  isActive    Boolean     @default(true)
  lastLoginAt DateTime?
  passwordResetToken String?
  passwordResetExpires DateTime?
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  // Relations
  collegeId   String?
  college     College?    @relation(fields: [collegeId], references: [id])
  refreshTokens RefreshToken[]

  @@index([email])
  @@index([collegeId])
}

```

```
model RefreshToken {
  id          String    @id @default(uuid())
  token       String    @unique
  expiresAt   DateTime
  revokedAt   DateTime?
  createdAt   DateTime  @default(now())

  userId String
  user   User   @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([token])
}

enum Role {
  STUDENT
  STAFF
  COORDINATOR
  ADMIN
}
```

Configuration

Environment Variables

```
# .env
# Database
DATABASE_URL="postgresql://user:pass@host:5432/db?sslmode=require"

# JWT
JWT_SECRET="your-super-secret-key-min-32-chars"
JWT_ISSUER="campus-os-auth"
JWT_AUDIENCE="campus-os-services"

# Server
AUTH_PORT=4300
NODE_ENV=development

# CORS
CORS_ORIGINS="http://localhost:3000,http://localhost:5173"

# Logging
LOG_LEVEL=debug # debug, info, warn, error
```

Production Recommendations

1. **JWT_SECRET**: Use a cryptographically random string (32+ characters)
 2. **DATABASE_URL**: Use connection pooling (e.g., Neon's pooler)
 3. **CORS_ORIGINS**: Whitelist only production domains
 4. **LOG_LEVEL**: Set to **info** or **warn** in production
 5. **HTTPS**: Always use HTTPS in production
 6. **Rate Limiting**: Implement with Redis
 7. **Monitoring**: Add APM (Application Performance Monitoring)
-

Changelog

v1.0.0 (December 2024)

- Initial release
- JWT authentication with access/refresh tokens
- RBAC with 4 roles (Student, Staff, Coordinator, Admin)
- Password hashing with bcrypt
- Token refresh with rotation
- Password reset flow
- Structured logging with Pino
- Input validation with Zod
- PostgreSQL with Prisma ORM